Grundlagen der künstlichen Intelligenz

Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas

Universidad de Granada

Grundlagen der künstlichen Intelligenz

Los Del DGIIM, losdeldgiim.github.io

Arturo Olivares Martos

Granada, 2026

Índice general

1. Introduction

Artificial Intelligence (AI) is a field that is rapidly evolving and has the potential to revolutionize various industries. Before delving into the technical aspects of AI, it is important to understand the concept of intelligence and how it can be measured.

1.1. Intelligence Tests

Scientists have tried to decide what differentiates AI from computer algorithms; i.e. what is the essence of intelligence? This question has been debated for decades, and there are different tests that have been proposed to determine whether a machine can be considered intelligent or not. The importance of these tests is shown in the Chinese Room thought experiment.

- Chinese Room Experiment:

  The Chinese Room thought experiment involves a person who does not understand Chinese sitting in a room with a set of instructions for how to respond to Chinese characters. The person receives Chinese characters as input and uses the instructions to produce appropriate responses, which are then sent back out of the room.

  From the outside, it may appear that the person in the room understands Chinese, but in reality, they are simply following a set of rules without any comprehension of the language. This thought experiment raises questions about whether a machine can truly understand language or if it is simply following a set of instructions.

Some of the most well-known tests include:

- Turing Test:

  In the Turing Test, a human evaluator interacts with both a machine and a human without knowing which is which. If the evaluator cannot reliably distinguish between the machine and the human, then the machine is said to have passed the Turing Test and is considered to be intelligent.

  It only tests for the ability to mimic human behavior, not for true understanding or consciousness.

- Winograd Test:

  The test involves a series of sentences known as "Winograd Schemas". They include ambiguous pronouns that require common sense and contextual understanding to resolve.

This test is not about factual knowledge but rather about understanding context.

- Lovelace Test:

  The Lovelace Test is designed to evaluate a machine's ability to create something original and surprising.

  This test is extremely difficult for machines to pass, as they often just combine existing ideas rather than creating something truly new.

- Metzinger Test:

  The Metzinger Test is designed to evaluate a machine's ability to have a subjective experience, a sense of "self, I", and consciousness. There is no definitive method for conducting the test in practice, since subjective experience cannot be observed from the outside.

1.2. Initial Definitions

From more general concepts to more specific ones, where each one builds upon the previous, some important terms in the field of AI include:

- Data Science: Field that turns data into knowledge and decisions.
- Artificial Intelligence: Field that aims to create machines that can perform tasks that typically require human intelligence.
- Machine Learning: Subfield of AI that focuses on training machines to learn from data and improve their performance over time without being explicitly programmed. There are three main types of machine learning:
  - Supervised Learning: The machine is trained on labeled data, where the correct output is provided for each input.
  - Unsupervised Learning: The machine is trained on unlabeled data, where it must find patterns and relationships on its own.
  - Reinforcement Learning: The machine learns by interacting with an environment and receiving feedback in the form of rewards or penalties based on its actions.
- Neural Networks: A type of (usually supervised) machine learning model inspired by the structure and function of the human brain. They consist of layers of interconnected nodes (neurons) that process and transmit information.
- Deep Learning: Neural networks with multiple layers. Deep learning has been particularly successful in tasks such as image recognition, natural language processing, and speech recognition.

2. Data Science

Data Science is a multidisciplinary field that combines statistical analysis, machine learning, and domain expertise to extract insights and knowledge from data.

2.1. Statistics

Data science involves, in a large scale, a lot of statistics, which is the branch of mathematics that deals with the collection, analysis, interpretation, presentation, and organization of data. Some of the most important concepts in statistics include:

- Outlier: A data point that is significantly different from the other data points in the dataset. Outliers can be caused by errors in data collection or by natural variability in the data. They can have a significant impact on the results of statistical analysis, so it is important to identify and handle them appropriately.

- Mean: The average of a set of numbers. It represents the central tendency of the data.

$$\text{Mean} = E[X] = \mu := \frac{1}{n}\sum_{i=1}^{n} x_i$$

- Median: The middle value of a set of numbers when they are arranged in order. It is less affected by outliers than the mean. Other terms as percentiles and quartiles are also used to describe the distribution of the data. The p-th percentile is the value below which $p\,\%$ of the data falls. The first quartile (Q1) is the 25th percentile, the second quartile (Q2) is the 50th percentile (which is also the median), and the third quartile (Q3) is the 75th percentile.

- Mode: The value that appears most frequently in a dataset. It is useful for categorical data.

- Variance: A measure of how much the data varies from the mean. It is calculated as the average of the squared differences from the mean.

$$\text{Variance} = E[(X-\mu)^2] = \sigma^2 := \frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2$$

  It can be however calculated using the Besel correction, which divides by $n-1$ instead of n to provide an unbiased estimator of the population variance when working with a sample. This is because when we calculate the sample mean,

we are using the same data points that we are trying to estimate the variance for, which can lead to an underestimation of the true population variance. By dividing by $n-1$, we are effectively increasing the variance estimate to account for this bias.

$$\text{Sample Variance} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \mu)^2$$

- Standard Deviation: The square root of the variance. It is a measure of how much the data varies from the mean, but it is in the same units as the data.

$$\text{Standard Deviation} = \sqrt{\text{Variance}} = \sigma$$

- Skewness: A measure of the asymmetry of the distribution of the data. A positive skew indicates that the data is skewed to the right, which means that the mean is greater than the median, meaning that there are more values on the right side of the distribution than on the left. A negative skew indicates that the data is skewed to the left, which means that the mean is less than the median, meaning that there are more values on the left side of the distribution than on the right. A skewness of zero indicates that the data is perfectly symmetrical.

- Kurtosis: A measure of the "tailedness.of the distribution of the data. A high kurtosis indicates that the data has heavy tails, which means that there are more extreme values in the data than would be expected in a normal distribution. A low kurtosis indicates that the data has light tails, which means that there are fewer extreme values in the data than would be expected in a normal distribution. A kurtosis of zero indicates that the data has the same tailedness as a normal distribution.

- Covariance: A measure of how much two random variables change together. It is calculated as:

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

  A positive covariance indicates that the two variables tend to increase or decrease together, while a negative covariance indicates that one variable tends to increase when the other decreases. A covariance of zero indicates that there is no linear relationship between the two variables.

- Correlation: A measure of the strength and direction of the linear relationship between two random variables. It is calculated as:

$$\text{Correlation}(X, Y) = r_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

  where σ_X and σ_Y are the standard deviations of X and Y, respectively. The correlation coefficient ranges from -1 to 1, where a value of 1 indicates a perfect positive linear relationship, a value of -1 indicates a perfect negative linear relationship, and a value of 0 indicates no linear relationship between the two variables.

It should be noted that correlation does not imply causation. Just because two variables are correlated does not mean that one variable causes the other to change. There may be other factors at play that are influencing both variables, or the correlation may be purely coincidental.

2.2. Data

Data is the raw material that fuels the development of AI systems, and understanding how to work with data is crucial for anyone interested in the field of AI. Two main people work on it:

- Data Engineer: Responsible for building and maintaining the infrastructure that allows for the collection, storage, and processing of data.
- Data Scientist: Responsible for analyzing and interpreting data to extract insights and knowledge.

When working with data, the PPDAC Data Cycle is often used as a framework to guide the process. The PPDAC Data Cycle consists of five stages:

- Problem: Defining the problem that needs to be solved, stating the objectives and questions that need to be answered.
- Plan: Developing a plan for how to answer the questions and achieve the objectives. This includes deciding on which data to collect, how to collect it, and how to analyze it.
- Data: Collecting the data according to the plan. This can involve gathering data from various sources, such as databases, APIs, or web scraping. It also involves preparing the data for analysis, which may include cleaning, transforming, and encoding the data.
- Analysis: Analyzing the data to extract insights and knowledge. This can involve using various statistical and machine learning techniques to identify patterns and relationships in the data.
- Conclusion: Drawing conclusions from the analysis and communicating the results. This can involve creating visualizations, writing reports, or presenting the findings to stakeholders.

2.2.1. Data Collection

Regarding data collection, there are some aspects that must be taken into account:

- Data Quality: The quality of the data is crucial for the success of any AI project.
- Data Integrity: Ensuring that the data is accurate and reliable.
- Data Consistency: Ensuring that the data is consistent across different sources and formats.

2.2.2. Data Preprocessing

Once the data has been collected, it must be preprocessed before it can be used for training AI models. This involves:

- Data Validation: Checking the data for errors and inconsistencies. Depending on the source and type of data, some rules can be applied to validate it.
- Data Sorting: Organizing the data in a way that makes it easier to work with. This can involve sorting the data by a specific column.
- Data Aggregation: Combining data from multiple sources or records into a single record. This can involve calculating summary statistics, such as the mean or median, for a group of records. This is often used to reduce the size of the data and to make it easier to analyze. However, it can also lead to loss of information, so it should be used with caution.
- Data Cleaning: Removing or correcting any errors or inconsistencies in the data.
- Data Transformation: Converting the data into a format that can be used for training AI models.

Data preprocessing also involves data scaling, which is the process of normalizing or standardizing the data to ensure that it is on the same scale. This is important because many AI algorithms are sensitive to the scale of the data and can perform poorly if the data is not scaled properly. Some common scalers include:

- `Binarizer`: Converts categorical data into binary data. A threshold is set, and values above the threshold are converted to 1, while values below the threshold are converted to 0.

$$X' = \begin{cases} 0 & \text{if } X \leqslant \text{threshold} \\ 1 & \text{if } X > \text{threshold} \end{cases}$$

- `MaxAbsScaler`: Scales the data to the range [-1, 1] by dividing each value by the maximum absolute value in the data.

$$X' = \frac{X}{\text{máx}(|X|)}$$

- `MinMaxScaler`: Scales the data to a specified range, let's say [mín, máx]. Firsrly the data is scaled to the range [0, 1] (X_{std}), and then it is scaled to the specified range.

$$X_{\text{std}} = \frac{X - \text{mín}(X)}{\text{máx}(X) - \text{mín}(X)} \qquad X' = X_{\text{std}} \cdot (\text{máx} - \text{mín}) + \text{mín}$$

- `Normalizer`: Scales the data to have a unit norm. This is done by dividing each value by the norm of the data. Usually, the L2 norm is used.

$$X' = \frac{X}{\|X\|}$$

- `PowerTransformer`: Applies a power transformation to the data to make it more Gaussian-like. This is useful for data that is not normally distributed.
- `RobustScaler`: Scales the data using statistics that are robust to outliers. This is useful for data that contains outliers.

$$X' = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

  where $\text{IQR}(X)$ is the interquartile range of X.
- `StandardScaler`: Scales the data to have a mean of 0 and a standard deviation of 1.

$$X' = \frac{X - \mu}{\sigma}$$

  where μ is the mean of X and σ is the standard deviation of X.

One important aspect of data preprocessing is *encoding* the data. This is the process of converting categorical data into numerical data that can be used for training AI models. There are several techniques for encoding data, such as:

- `Ordinal Encoding`: Assigning a unique integer to each category in the data. The order of the integers is important, as it implies a relationship between the categories.

$$\{\text{small}, \text{medium}, \text{large}\} \to \{0, 1, 2\}$$

- `Label Encoding`: Similar to ordinal encoding, but it does not imply any relationship between the categories. Each category is assigned a unique integer, but the order of the integers does not matter.

$$\{\text{red}, \text{green}, \text{blue}\} \to \{0, 1, 2\}$$

- `One-Hot Encoding`: Creating a binary column for each category in the data.

$$\{\text{red}, \text{green}, \text{blue}\} \to \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- `Dummy Encoding (Leave out the last)`: Similar to one-hot encoding, but it leaves out the last category to avoid multicollinearity.

$$\{\text{red}, \text{green}, \text{blue}\} \to \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

- `Absolute Frequency Encoding`: Assigning a unique integer to each category based on the absolute frequency of the category in the data. Once drawback of this technique is that, if two categories have the same frequency, they will be assigned the same integer, which can lead to confusion.

$$\{\text{red}, \text{red}, \text{red}, \text{green}, \text{blue}, \text{blue}\}$$
$$\{\text{red}, \text{green}, \text{blue}\} \to \{3, 1, 2\}$$

- `Relative Frequency Encoding`: Assigning a unique integer to each category based on the relative frequency of the category in the data.

$$\{\text{red}, \text{red}, \text{red}, \text{green}, \text{blue}, \text{blue}\}$$
$$\{\text{red}, \text{green}, \text{blue}\} \rightarrow \{3/6, 1/6, 2/6\}$$

- `Feature Hash Encoding`: Hashing the categories into a fixed number of columns. This is useful for data with a large number of categories.

2.2.3. Data Usage

Once the data has been preprocessed, they can be used. There are two main destinations for the data:

- Data Visualization: The process of creating visual representations of data to help understand and communicate insights. This is used by people.

  The Gestalt Principles of Perception are a set of principles that describe how people perceive and organize visual information, and they can be applied to data visualization to create more effective and engaging visualizations.

- Data Modeling: The process of using data to train AI models. This is used by machines.

3. Search Algorithms

In our current world, plenty of algorithms are used to search for information. From the most basic ones, such as sorting algorithms, to the most complex ones, such as search engines. In this chapter, we will explore some of the most common search algorithms and their applications.

3.1. Graph Theory

Search algorithms are often used in graphs, as they provide a way to represent complex relationships between data. A graph $G = (V, E)$ is defined as a pair of sets, where V is called the set of vertices and E is called the set of edges. The vertices represent the data points, while the edges represent the relationships between them. There are different types of graphs, such as directed and undirected graphs, weighted and unweighted graphs, or cyclic and acyclic graphs. Each type of graph has its own properties and algorithms that can be applied to it. An example graph is shown in Figure 3.1.

A graph is usually represented as an *adjacency matrix*. Where the rows represent the origin vertices, the columns represent the destination vertices[1], and the values a_{ij} represent:

- If the graph is unweighted, a_{ij} represents the number of edges from vertex i to vertex j.
- If the graph is weighted, a_{ij} represents the weight of the edge from vertex i to vertex j. If there is no edge, a_{ij} is usually set to infinity or an invalid value.

If there are a lot of vertices that are not connected to each other, the adjacency matrix can be very sparse, meaning that it contains a lot of zeros. In this case, it is

[1]This could also be the opposite: the origin vertices could be the colums and the destination vertices, the rows.

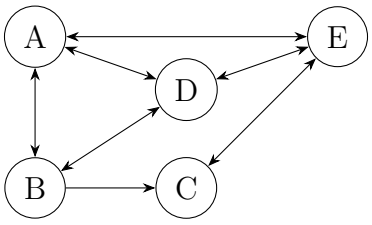

Figura 3.1: Example of a directed graph.

```
adjacency_list = {
    'A': ['B', 'D', 'E'],
    'B': ['A', 'C', 'D'],
    'C': ['B', 'E'],
    'D': ['A', 'B', 'E'],
    'E': ['A', 'C', 'D']
}
```

Código fuente 1: Adjacency list for the graph in Figure 3.1.

more efficient to use an *adjacency list* to represent the graph, where each vertex is represented as a list of its neighboring vertices. This can save a lot of space, especially for large graphs with few edges. For example, the adjacency matrix for the graph in Figure 3.1 is shown in Equation 3.1, while the adjacency list is Listing 1.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix} \tag{3.1}$$

This matrix is also called "next-neighbor" matrix, as the neighboring vertices of a vertex i can be found by looking at the i-th row of the matrix. The "next-next-neighbor" matrix, on the other hand, is obtained by multiplying the adjacency matrix by itself. This matrix can be used to find the vertices that are two steps away from a given vertex.

Lastly, the *transition matrix* is obtained by normalizing the adjacency matrix. This is done by dividing each entry a_{ij} by the sum of the entries in the i-th row. The transition matrix can be used to find the probability of transitioning from one vertex to another in a random decision process.

3.1.1. PageRank

PageRank is an algorithm used to determine the importance of nodes in a graph. It is well-known for being used by Google Search to rank web pages in their search engine results. It works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites. Everything is represented as a directed graph, where the vertices represent web pages and the edges represent hyperlinks between them. There are certain parameters that can be adjusted in the PageRank algorithm, such as:

- The teleportation probability vector. It's n-th entry represents the probability of jumping to the n-th page when a random surfer decides to teleport (going to a random page instead of following a link). It is usually set to a uniform distribution, meaning that the random surfer has an equal probability of jumping to any page.

- The damping factor α, which represents the probability that a random surfer will continue following links rather than teleporting.
- The convergence threshold, which determines when the algorithm should stop iterating and return the PageRank values.

The algorithm returns the PageRank vector, whose n-th entry represents the PageRank of the n-th page. The PageRank values decide in which order the pages are shown in the search results, with higher PageRank values indicating more important pages. The PageRank algorithm works as follows:

1. Initialize the PageRank vector with the teleportation probability vector.
2. Update the old PageRank vector (PR) by adding the transition matrix multiplied by the old PageRank vector (weighted by the damping factor α) and the teleportation probability vector (weighted by $1 - \alpha$):

   $$PR = \alpha \cdot M \cdot PR + (1 - \alpha) \cdot v$$

   Where M is the transition matrix and v is the teleportation probability vector.
3. Repeat the process until the change in the PageRank vector is less than the convergence threshold (or a maximum number of iterations is reached).

Once the algorithm converges, the PageRank values can be used to rank the web pages in search results. The higher the PageRank value, the more important the page is considered to be, and therefore, it will be ranked higher in the search results.

3.2. Sorting Algorithms

Sorting algorithms are a fundamental part of computer science, as they allow us to organize data in a specific order. In addition, they are used in many search algorithms, as they can help to reduce the search space and improve the efficiency of the search.

3.2.1. Insertion Sort

This algorithm works by dividing the array into a sorted and an unsorted part. The sorted part is initially empty, and the unsorted part contains all the elements.

1. Start with the first element of the unsorted part and compare it with the last element of the sorted part.
2. IMove all the elements of the sorted part that are greater than the current element to the right.
3. Insert the current element into its correct position in the sorted part.
4. Repeat the process for all the elements in the unsorted part until it is empty.

It has a time complexity of $O(n^2)$.

3.2.2. Bubble Sort

This algorithm works by repeatedly swapping adjacent elements if they are in the wrong order. The process is repeated until the array is sorted.

1. Compare each pair of adjacent elements in the array.
2. If the elements are in the wrong order, swap them.
3. Repeat the process for all the elements in the array until it is sorted.

It has a time complexity of $O(n^2)$.

3.2.3. Shaker Sort

This algorithm is a variation of Bubble Sort that sorts in both directions on each pass through the list. It works by comparing adjacent elements and swapping them if they are in the wrong order, similar to Bubble Sort. However, it also compares elements in the opposite direction, which can help to reduce the number of passes needed to sort the array.

1. Start at the beginning of the array and compare each pair of adjacent elements.
2. If the elements are in the wrong order, swap them.
3. After reaching the end of the array, reverse the direction and compare each pair of adjacent elements again.
4. Repeat the process until the array is sorted.

It has a time complexity of $O(n^2)$, but with lower constant factors than Bubble Sort.

3.2.4. Quick Sort

This divide-and-conquer algorithm works by selecting a pivot element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. This process continues until the base case of an empty array or an array with a single element is reached.

1. Choose a pivot element from the array.
2. Partition the array into two sub-arrays: one with elements less than the pivot and one with elements greater than the pivot.
3. At this point, the pivot is in its final position.
4. Recursively apply the above steps to the sub-arrays until the base case is reached.

The efficiency of Quick Sort depends on the choice of the pivot. The complexity depends on the partitioning of the array. If the pivot divides the array into two equal halves, the time complexity is $O(n \log n)$. However, if the pivot is consistently the smallest or largest element, the time complexity degrades to $O(n^2)$.

3.2.5. Topological Sorting: Kahn's Algorithm

This algorithm is used to sort the vertices of a directed acyclic graph (DAG) in a linear order. If there is a directed edge from vertex u to vertex v, then u comes before v in the ordering. It is used in several applications, such as scheduling tasks, resolving dependencies, and determining the order of compilation in programming languages. The algorithm works as follows:

1. The result is going to be stored in a list called L.
2. Find all vertices with no incoming edges and add them to a set S.
3. While S is not empty:
   a) Remove a vertex n from S and add it to L.
   b) For each vertex m with an edge from n to m:
      1) Remove the edge from n to m.
      2) If m has no other incoming edges, add m to S.
4. If there are still edges in the graph, it means that there is a cycle, and therefore, a topological sort is not possible. Otherwise, the list L contains the vertices in topologically sorted order.

3.3. Uninformed Search

Uninformed search algorithms are those that do not have any additional information about the search space. They explore the search space in a systematic way, without any guidance or heuristics.

3.3.1. Breadth-First Search

This algorithm explores the search space level by level, starting from the root node and expanding all the nodes at the present depth before moving on to the nodes at the next depth level. It uses a queue (FIFO) to keep track of the nodes to be explored. It is guaranteed to find the shortest path, but it can be memory intensive, as it needs to store all the nodes at the current depth level.

3.3.2. Depth-First Search

This algorithm explores the search space by going as deep as possible along each branch before backtracking. It uses a stack (LIFO) to keep track of the nodes to be explored. It is not guaranteed to find the shortest path, but it can be more memory efficient than breadth-first search.

3.3.3. Depth-Limited Search

This algorithm is a variation of depth-first search that limits the depth of the search. It is used to avoid infinite loops in cases where the search space is infinite or has cycles. It uses a stack (LIFO) to keep track of the nodes to be explored, and it keeps track of the current depth of the search. If the depth limit is reached, it backtracks and explores other branches. It is not guaranteed to find the searched node, but it can be more memory efficient than depth-first search. The choice of the depth limit is crucial, as it can affect the completeness and optimality of the search.

3.3.4. Iterative Deepening Search

This algorithm combines the benefits of both breadth-first and depth-first search. It performs a depth-limited search with increasing depth limits until a solution is found.

1. Start with a depth limit of 0.
2. Perform a depth-limited search with the current depth limit.
3. If a solution is found, return it. Otherwise, increase the depth limit and repeat the process.

A stack (LIFO) is used to keep track of the nodes to be explored in the depth-limited search. It is guaranteed to find the shortest path, and it can be more memory efficient than breadth-first search.

3.3.5. Bidirectional Search

This algorithm is used to find the shortest path between two nodes in a graph. It works by simultaneously searching (typically BFS) from both the start node and the goal node until the two searches meet in the middle. It uses two data structures to keep track of the nodes to be explored from both directions. It is guaranteed to find the shortest path, and it can be more efficient than unidirectional search, especially in cases where the search space is large.

3.3.6. Uniform Cost Search

This algorithm is used in weighted graphs (where the edges have positive costs) to find the least costly path from the root node to a goal node. It is similar to breadth-first search, but it takes into account the cost of the path. It uses a priority queue to keep track of the nodes to be explored, where the priority is determined by the cost of the path from the root node to the current node. It is guaranteed to find the least costly path, but it can be memory intensive, as it needs to store all the nodes in the search space.

1. Start with the root node and add it to the priority queue with a cost of 0.
2. While the priority queue is not empty:
    a) Remove the node with the lowest cost from the priority queue.

b) If the node is a goal node, return the path to it.

c) Otherwise, expand the node and add its children to the priority queue with their respective cumulative costs.

Local Uniform Cost Search

This algorithm is a variation of uniform cost search that is used to find the least costly path from a given node to a goal node, but just considering the local neighborhood of the given node. For each selected node, the next selected node is the one with the lowest cost among its neighbors. It is not guaranteed to find the least costly path, but it can be more efficient than uniform cost search in cases where the search space is large and the goal node is close to the given node.

1. Start with the given node.
2. While the goal node has not been reached, or a maximum number of iterations has not been reached:
   a) Evaluate the neighbors of the current node and select the one with the lowest cost.
   b) Move to the selected neighbor and repeat the process.

3.3.7. Dijkstra's Algorithm

This algorithm is a specific implementation of uniform cost search that is used to find the shortest path from a source node to all other nodes in a weighted graph. It works by maintaining a priority queue of nodes to be explored, where the priority is determined by the cumulative cost from the source node to the current node. The algorithm iteratively expands the node with the lowest cumulative cost and updates the costs of its neighbors until all nodes have been explored. It is guaranteed to find the shortest path from the source node to all other nodes, but it can be memory intensive, as it needs to store all the nodes in the search space.

1. Start with the source node and add it to the priority queue with a cost of 0. Every other node is added to the priority queue with a cost of infinity.
2. While the priority queue is not empty (it is not empty until all nodes have been explored):
   a) Remove the node with the lowest cumulative cost from the priority queue.
   b) For each neighbor of the current node:
      1) Calculate the cumulative cost to reach the neighbor through the current node.
      2) If this cumulative cost is less than the previously known cost for that neighbor, update the cost and modify it in the priority queue.

It should be noted that Dijkstra's algorithm is the same as UCS, but it simply does not stop when it finds a goal node, but rather continues to explore until all nodes have been explored. This is because Dijkstra's algorithm is designed to find

the shortest path from a source node to all other nodes in the graph, while UCS is designed to find the least costly path from a root node to a specific goal node. Therefore, Dijkstra's algorithm is more general than UCS, but also more computationally intensive.

3.4. Informed Search

Informed search algorithms are those that have additional information about the search space. They use this information to guide the search and improve its efficiency. The additional information is usually in the form of a heuristic function, which estimates the cost of reaching the goal from a given node.

3.4.1. Greedy Best First Search

This algorithm uses a priority queue to keep track of the nodes to be explored, where the priority is determined by the heuristic function. It always expands the node that is estimated to be closest to the goal.

1. Start with the root node and add it to the priority queue with a priority determined by the heuristic function.
2. While the priority queue is not empty:
   a) Remove the node with the lowest heuristic value from the priority queue.
   b) If the node is a goal node, return the path to it.
   c) Otherwise, expand the node and add its children to the priority queue with their respective heuristic values.

The efficiency of Greedy Best First Search depends on the quality of the heuristic function. The main drawback of this algorithm is that it is not guaranteed to find the optimal solution, as it does not consider the actual cost of the path, but only its estimated cost to the goal.

3.4.2. A* Search

This algorithm is a combination of uniform cost search and greedy best first search. It uses a priority queue to keep track of the nodes to be explored, where the priority is determined by:

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the actual cost from the root node to the current node n, and $h(n)$ is the heuristic estimate of the cost from n to the goal. Two important properties of the heuristic function are:

- Admissible: It never overestimates the cost to reach the goal. It can be expressed as:
  $$h(n) \leqslant h^*(n) \qquad \forall n \in V$$
  where $h^*(n)$ is the true cost from node n to the goal. If the heuristic is admissible, A* Search is guaranteed to find the optimal solution.

- Consistent (or monotonic): For every node n and every successor n' of n, the estimated cost of reaching the goal from n is no greater than the cost of getting from n to n' plus the estimated cost of reaching the goal from n'. Formally, this can be expressed as:

$$h(n) \leqslant c(n, n') + h(n')$$

where $c(n, n')$ is the actual cost of moving from node n to node n'. If the heuristic is consistent, it is also admissible, and A* Search is guaranteed to find the optimal solution. In addition, if the heuristic is consistent, A* Search is more efficient, as it does not need to revisit nodes that have already been explored.

Therefore, the A* Search algorithm is:

1. Start with the root node and add it to the priority queue with a priority determined by $f(n) = g(n) + h(n)$. All other nodes are added to the priority queue with a priority of infinity.
2. While the priority queue is not empty:
   a) Remove the node with the lowest $f(n)$ value from the priority queue.
   b) If the node is a goal node, return the path to it.
   c) Otherwise, expand the node. For each child of the current node:
      1) Calculate $g(n')$ as $g(n) + c(n, n')$, where $c(n, n')$ is the actual cost of moving from node n to node n'. If that new $g(n')$ value is less than the previously known $g(n')$ value for that child, update $g(n')$ and calculate $f(n') = g(n') + h(n')$.

3.5. Local Search

Local search algorithms are those that operate on a single solution and try to find a better solution by making small changes to it. They are often used in optimization problems. Therefore, the aim is finding a local optimum, not necessarily a global one.

3.5.1. Hill Climbing

This algorithm starts with an arbitrary solution and iteratively makes small changes to it, accepting the change if it results in a better solution. The process continues until no further improvements can be made (or a maximum number of iterations is reached).

1. Start with an arbitrary solution.
2. Generate the neighbors of the current solution by making small changes to it.
3. Evaluate the neighbors and select the one with the best value.
4. If the best neighbor is better than the current solution, move to that neighbor and repeat the process. Otherwise (if no neighbor is better), return the current solution as a local optimum.

3.5.2. Simulated Annealing

This algorithm is a variation of hill climbing that allows for occasional moves to worse solutions in order to escape local optima. It is inspired by the annealing process in metallurgy, where a material is heated and then slowly cooled to remove defects and improve its properties. The algorithm works as follows:

1. Start with an arbitrary solution, an initial temperature $T \geqslant 0$, and a cooling rate α (where $0 < \alpha < 1$).
2. Generate a neighbor of the current solution by making a small change to it.
3. Evaluate the neighbor and calculate the change in the objective function ΔE (difference between the neighbor's value and the current solution's value).
4. If $\Delta E < 0$ (the neighbor is better), move to that neighbor. Otherwise, move to the neighbor with a probability of $e^{-\Delta E/T}$. Initially, with a high temperature, the algorithm is more likely to accept worse solutions, allowing it to explore the search space more broadly. As the temperature decreases, the algorithm becomes less likely to accept worse solutions, focusing more on local improvements.
5. Update the temperature by multiplying it by the cooling rate: $T = \alpha T$.
6. Repeat the process until a stopping criterion is met (e.g., a maximum number of iterations, a minimum temperature, or a satisfactory solution).

This algorithm doesn't necessarily find the optimal solution, but it can avoid being stuck in local optima, especially if the cooling schedule is well-designed.

4. Übungen

4.0. Introduction

The Python part of this exercise is available in this Jupyter Notebook.

Intelligence Tests

Ejercicio 1. Turing Test vs. Lovelace Test: Which test focuses on creativity and which on simulating conversation? How do these two tests work?

The Turing Test focuses on simulating conversation. It works by having a human evaluator engage in a natural language conversation with both a machine and a human without knowing which is which. If the evaluator cannot reliably distinguish the machine from the human, the machine is considered to have passed the Turing Test.

The Lovelace Test, on the other hand, focuses on creativity. It requires an AI to create something original (like a piece of art, music, or a story) that cannot be attributed to any human creator. The AI must be able to explain how it created the work and why it is original, demonstrating a level of creativity that goes beyond mere imitation.

Ejercicio 2. Why do many simple chatbots fail the Winograd Schema?

Many simple chatbots fail the Winograd Schema because they lack the necessary contextual understanding and common sense reasoning required to correctly interpret the ambiguous sentences used in the test. The Winograd Schema consists of sentences that have a pronoun with an ambiguous antecedent, and the correct interpretation depends on understanding the context and the relationships between the entities involved. Simple chatbots often rely on pattern matching or statistical methods rather than true comprehension, which leads to incorrect answers when faced with such nuanced language challenges.

Ejercicio 3. What is the main idea behind the Metzinger test for checking if an AI is self-aware, in simple terms? What is one thing that might make this test difficult to use for all types of AI?

The main idea behind the Metzinger test is to check if an AI can understand and describe its own internal processes and experiences, which would indicate self-awareness. One difficulty with this test is that it relies on the AI's ability to communicate its thoughts and feelings, which may not be possible for all types of AI,

especially those that do not have a natural language processing capability or a way to express their internal states effectively. Therefore, there is no universally applicable method to determine if an AI is self-aware, as it may depend on the specific design and capabilities of the AI in question.

4.1. Data Processing / Cleaning

The Python part of this exercise is available in this Jupyter Notebook.

Data Filtering and Transforming

Ejercicio 4.1.1. Imagine you have a list of all the students in a school. You only want to look at the students in the 5th grade (data filtering). Then, you want to calculate the average age of those 5th graders (data transforming). Explain why you need both steps.

We first need to filter the data to focus only on the students in the 5th grade, as we are specifically interested in that group. This step allows us to narrow down our dataset to the relevant subset of students. After filtering, we need to transform the data by calculating the average age of those 5th graders. This transformation step is necessary to derive meaningful insights from the filtered data, as it provides us with a summary statistic (the average age) that can help us understand the characteristics of the 5th-grade students. Without filtering, we would be calculating the average age of all students, which would not give us the specific information we are looking for about the 5th graders.

Ejercicio 4.1.2. You have a list of toys that were sold in a store, including the toy's name and the price it sold for.

1. Give an example of a question you could answer by only looking at toys that cost more than 20 (data filtering).

   A question that could be answered by data filtering would be: "Which toys that cost more than 20 euros were sold in the last month?" (Here, we filter the data by price greater than 20 euros and potentially by the sales date).

2. Give an example of something you could calculate or change about the toy prices (data transforming).

   An example of data transformation would be calculating the average selling price of all toys or converting all prices from euros to another currency (e.g., US dollars) based on a specific exchange rate. Another transformation could be creating a new column indicating whether a toy is classified as "cheap" (e.g., $<$ 15 euros), "mid-priced" (e.g., 15 euros $-$ 30 euros), or "expensive" ($>$ 30 euros) based on the selling price.

Data Understanding

Ejercicio 4.1.3. What is the difference between quantitative (numerical) and qualitative (categorical) data?

Quantitative data refers to numerical values that can be measured and quantified, such as height, weight, or age. Qualitative data, on the other hand, refers to categorical values that describe characteristics or attributes, such as color or gender. Quantitative data can be analyzed using mathematical operations, while qualitative data is typically analyzed using descriptive statistics or categorization.

Ejercicio 4.1.4. What does the abbreviation `NaN` mean in a dataset?

`NaN` stands for "Not a Number" and is used in datasets to represent missing or undefined values. It is also used to indicate that a value is not a valid number, such as the result of an undefined mathematical operation (e.g., division by zero). In data analysis, `NaN` values need to be handled appropriately, either by filling them with a specific value or dropping them.

Ejercicio 4.1.5. Briefly explain the difference between the mean and the median. Which value is more sensitive to outliers?

The mean is the average of a set of numbers, calculated by summing all the values and dividing by the count of values. The median is the middle value in a sorted list of numbers. The mean is more sensitive to outliers because it can be significantly affected by extremely high or low values, while the median is less affected as it only considers the middle value(s) and not the magnitude of all values.

Ejercicio 4.1.6. Calculate the quartiles for the following dataset: [10, 9, 1, 4, 3, 2, 5, 6, 6, 6, 3, 9].

First, we need to sort the dataset:

$$[1, 2, 3, 3, 4, 5, 6, 6, 6, 9, 9, 10]$$

The quartiles are then:

$$Q2 = \frac{6+5}{2} = 5{,}5$$
$$Q1 = \frac{3+3}{2} = 3$$
$$Q3 = \frac{6+9}{2} = 7{,}5$$

Data Cleaning

Ejercicio 4.1.7. Name two reasons why data is often dirty in the real world.

Data can be dirty due to:

- Human error during data entry, which can lead to typos, incorrect values, or missing data.
- Inconsistent data formats or structures, such as different date formats or varying units of measurement across datasets.

Ejercicio 4.1.8. What happens when you apply the `dropna()` function to a DataFrame?

The `dropna()` function removes any rows (or columns, if specified with the `axis=1` parameter) that contain missing values (`NaN`) from the DataFrame. The original DataFrame is not modified unless the `inplace=True` parameter is used.

This is a common data cleaning step to ensure that the dataset does not contain incomplete records that could affect analysis or modeling.

Ejercicio 4.1.9. Why do you scale (normalize) numerical data before feeding it into a machine learning model?

Scaling (normalizing) numerical data is important because many machine learning algorithms are sensitive to the scale of the input features. If the features have different ranges, the algorithm may give more weight to those with larger values, leading to biased results. Scaling helps to ensure that all features contribute equally to the model's learning process and can improve the convergence speed and performance of the model.

Ejercicio 4.1.10. Name two possible encodings and explain them using an example.

They are explained in the page 11.

Ejercicio 4.1.11. Describe three common challenges encountered during data preparation and explain a technique to mitigate each challenge.

- Missing Data: This can be mitigated by using techniques such as imputation (filling in missing values with the mean, median, or mode) or by dropping rows/columns with a high percentage of missing values.
- Outliers: Outliers can skew the results of analysis and modeling. They can be mitigated by using methods such IQR to identify and either remove or transform outliers.
- Inconsistent Data: Inconsistent data can arise from different formats or units. This can be mitigated by standardizing the data format (e.g., converting all dates to a common format) and ensuring that all measurements are in the same units (e.g., converting all weights to kilograms).

Ejercicio 4.1.12. Explain the difference between data cleaning and data transformation. Provide an example of each.

Data Cleaning Data cleaning involves identifying and correcting errors or inconsistencies in the dataset to ensure that it is accurate and reliable. For example, removing duplicate entries from a customer database or filling in missing values in a sales dataset.

Data Transformation Data transformation involves converting data from one format or structure to another to make it suitable for analysis or modeling. For example, normalizing numerical features to a common scale or encoding categorical variables into numerical format using one-hot encoding.

Ejercicio 4.1.13. Why is it important to handle missing data appropriately? Discuss three different strategies for dealing with missing values and their potential drawbacks.

Handling missing data appropriately is crucial because it can significantly impact the results of analysis and modeling. If not addressed, missing data can lead to biased estimates, reduced statistical power, and inaccurate predictions. Three strategies for dealing with missing values are:

- Imputation: Filling in missing values with a specific value (e.g., mean, median, mode) can help maintain the dataset's size and structure. However, it can introduce bias if the imputed values do not accurately reflect the underlying distribution of the data.
- Deletion: Removing rows or columns with missing values can be simple but may lead to loss of information and reduced sample size.
- Model-based Imputation: Using statistical models to predict and fill in missing values based on other available data. This approach can be more accurate but is computationally intensive and may introduce additional complexity in model interpretation.

Ejercicio 4.1.14. What are outliers in a dataset and why can they be problematic? Describe three methods for detecting outliers.

Outliers are data points that significantly differ from the majority of the data. They can be problematic because they can skew the results of analysis and modeling, leading to inaccurate conclusions. Three methods for detecting outliers are:

- Z-score: This method identifies outliers by calculating the number of standard deviations a data point is from the mean. A common threshold is a Z-score of 3 or -3.
- Interquartile Range (IQR): This method identifies outliers by calculating the IQR (the difference between the third quartile and the first quartile) and defining outliers as data points that fall below $Q1 - 1{,}5 \cdot IQR$ or above $Q3 + 1{,}5 \cdot IQR$.
- Visual Methods: Box plots and scatter plots can visually reveal outliers in the data, allowing for easy identification of data points that deviate from the overall pattern.

Ejercicio 4.1.15. Explain the concept of data normalization or standardization. When would you prefer one over the other, and why is this a crucial step in many machine learning workflows?

Data normalization (scaling to a range) and standardization (scaling to have a mean of 0 and a standard deviation of 1) are techniques used to rescale numerical features. Normalization is preferred when the data does not follow a Gaussian distribution and when the algorithm does not assume any distribution. Standardization is preferred when the data follows a Gaussian distribution or when the algorithm assumes that the data is normally distributed. This step is crucial in machine learning workflows because it ensures that all features contribute equally to the model's learning process and can improve the convergence speed and performance of the model.

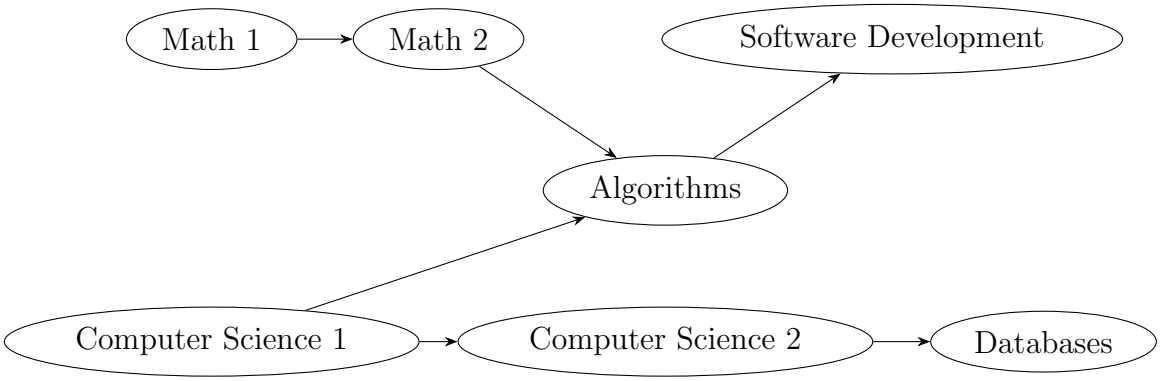

Figura 4.1: Directed graph representing the dependencies between lectures.

4.2. Search Algorithms

The Python part of this exercise is available in this Jupyter Notebook.

Kahn's Algorithm (Topological Sorting)

You are responsible for the course planning at a university. For a specific module, there are several lectures that partially build upon each other. Your task is to find an order in which all lectures can be attended without attending a lecture before one of its prerequisites.

Lectures (Nodes): Math 1, Math 2, Computer Science 1, Computer Science 2, Databases, Algorithms, Software Development

Dependencies (directed edges "must be attended before"):

- Math 1 → Math 2
- Computer Science 1 → Computer Science 2
- Computer Science 1 → Algorithms
- Math 2 → Algorithms
- Computer Science 2 → Databases
- Algorithms → Software Development

Ejercicio 4.2.1. Create the graph. Represent the directed graph as an adjacency list, and for each node, note the in-degree (number of incoming edges).

The graph is depicted in Figure 4.1. The adjacency list and in-degrees are as follows:

- Math 1: Adjacent to [Math 2], In-degree = 0
- Math 2: Adjacent to [Algorithms], In-degree = 1
- Computer Science 1: Adjacent to [CS 2, Algorithms], In-degree = 0
- Computer Science 2: Adjacent to [Databases], In-degree = 1
- Databases: Adjacent to [], In-degree = 1

Tabla 4.1: Steps of Kahn's Algorithm for topological sorting of the lectures.

Step	Final Order	New In-degrees
0	[]	Math 1: 0 Math 2: 1 Computer Science 1: 0 Computer Science 2: 1 Databases: 1 Algorithms: 2 Software Development: 1
1	[Math 1]	Math 2: 0 Computer Science 1: 0 Computer Science 2: 1 Databases: 1 Algorithms: 2 Software Development: 1
2	[Math 1, Math 2]	Computer Science 1: 0 Computer Science 2: 1 Databases: 1 Algorithms: 1 Software Development: 1
3	[Math 1, Math 2, Computer Science 1]	Computer Science 2: 0 Databases: 1 Algorithms: 0 Software Development: 1
4	[M1, M2, CS1, CS2]	Databases: 0 Algorithms: 0 Software Development: 1
5	[M1, M2, CS1, CS2, Databases]	Algorithms: 0 Software Development: 1
6	[M1, M2, CS1, CS2, Databases, Algorithms]	Software Development: 0
7	[M1, M2, CS1, CS2, Databases, Algorithms, SD]	

- Algorithms: Adjacent to [Software Development], In-degree = 2
- Software Development: Adjacent to [], In-degree = 1

Ejercicio 4.2.2. Apply Kahn's Algorithm. Perform Kahn's algorithm manually (or implement it in a programming language of your choice) to find a topological sorting of the lectures. Document each step:

1. Start with all nodes whose in-degree = 0.
2. Remove one of these nodes, add it to the order, and reduce the in-degrees of its neighbors.
3. Repeat until all nodes are processed.

The algorithm can be seen in the Table 4.1.

Ejercicio 4.2.3. Interpret the result. State the calculated order of the lectures, and determine if this order is unique. Justify your answer.

The calculated order of the lectures is:

Math 1 → Math 2 → Computer Science 1 → Computer Science 2 → Databases → Algorithms → Software Development

This order is not unique. For example, Computer Science 1 and Math 1 both have an in-degree of 0 at the start, so we could have chosen Computer Science 1 before Math 1, which would lead to a different valid topological order. The presence of multiple nodes with in-degree 0 at various steps allows for multiple valid topological sorts.